

# SKILL WEEK – 5

TASK1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir -p ~/Skill5
cd ~/Skill5
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ nano task1.c
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ gcc task1.c -o task1
./task1
Current directory: /home/shreyaj/Skill5
Changing to /tmp...
New directory: /tmp
Previous directory: /home/shreyaj/Skill5
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ |
```

```
GNU nano 8.7.1 task1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main() {
    char current[PATH_MAX];
    char previous[PATH_MAX];

    getcwd(current, sizeof(current));
    strcpy(previous, current);

    printf("Current directory: %s\n", current);
    printf("Changing to /tmp...\n");

    if (chdir("/tmp") == 0) {
        strcpy(previous, current);
        getcwd(current, sizeof(current));
        printf("New directory: %s\n", current);
        printf("Previous directory: %s\n", previous);
    } else {
        perror("cd failed");
    }

    return 0;
}
```

TASK2: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir -p ~/Skill5
shreyaj@LAPTOP-FT6VVM4D: ~/Skill5$ nano task2.c
shreyaj@LAPTOP-FT6VVM4D: ~/Skill5$ gcc task2.c -o task2
shreyaj@LAPTOP-FT6VVM4D: ~/Skill5$ ./task2
myshell> pwd
/home/shreyaj/Skill5
myshell> cd
cd: missing argument
myshell> cd /tmp
myshell> pwd
/tmp
myshell> cd
cd: missing argument
myshell> hello
myshell: invalid command
myshell> exit
Exiting shell...
shreyaj@LAPTOP-FT6VVM4D: ~/Skill5$ nano task
```

```
GNU nano 8.7.1 task2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void pwd_command() {
    char path[1024];
    if (getcwd(path, sizeof(path)) != NULL)
        printf("%s\n", path);
    else
        perror("pwd");
}

void cd_command(char *path) {
    if (path == NULL) {
        printf("cd: missing argument\n");
        return;
    }
    if (chdir(path) != 0)
        perror('cd');
}

void exit_command() {
    printf("Exiting shell...\n");
    exit(0);
}

int main() {
    char input[100];
    char *command;
    char *argument;

    while (1) {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0'; // Remove newline

        command = strtok(input, " ");
        argument = strtok(NULL, " ");

        if (command == NULL)
            continue;

        if (strcmp(command, "pwd") == 0) {
            pwd_command();
        } else if (strcmp(command, "cd") == 0) {
            cd_command(argument);
        } else if (strcmp(command, "exit") == 0) {
            exit_command();
        } else {
            printf("myshell: invalid command\n");
        }
    }

    return 0;
}
```

```
int main() {
    char input[100];
    char *command;
    char *argument;

    while (1) {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0'; // Remove newline

        command = strtok(input, " ");
        argument = strtok(NULL, " ");

        if (command == NULL)
            continue;

        if (strcmp(command, "pwd") == 0) {
            pwd_command();
        } else if (strcmp(command, "cd") == 0) {
            cd_command(argument);
        } else if (strcmp(command, "exit") == 0) {
            exit_command();
        } else {
            printf("myshell: invalid command\n");
        }
    }

    return 0;
}
```

TASK3: To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
shreyaj@LAPTOP-FT6VVM4D:~$ mkdir -p ~/Skill5
cd ~/Skill5
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ nano task3.c
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ gcc task3.c -o task3
./task3
Testing pwd command:
Current Directory: /home/shreyaj/Skill5

Testing exit request...
Resources cleaned successfully.
Program exiting...
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ |
```

```
shreyaj@LAPTOP-FT6VVM4D: × + ▾
GNU nano 8.7.1 task3.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>

void show_current_directory() {
    char path[PATH_MAX];
    if (getcwd(path, sizeof(path)) != NULL) {
        printf("Current Directory: %s\n", path);
    } else {
        perror("getcwd");
    }
}

int main() {
    printf("Testing pwd command:\n");
    show_current_directory();

    printf("\nTesting exit request...\n");
    printf("Resources cleaned successfully.\n");
    printf("Program exiting...\n");

    return 0;
}
```

TASK4: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
shreyaj@LAPTOP-FT6VVM4D: × + ▾
shreyaj@LAPTOP-FT6VVM4D:~$ cd ~/Skill5
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ task4.c
task4.c: command not found
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ nano task4.c
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ gcc task4.c -o task4
./task4
Variable exported successfully.
STUDENT = Shreyaj
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ nano child.c
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ nano parent.c
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ gcc child.c -o child
gcc parent.c -o parent
./parent
Parent: STUDENT = Shreyaj
Child process received STUDENT = Shreyaj
shreyaj@LAPTOP-FT6VVM4D:~/Skill5$ |
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v task4.c
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int is_valid_name(const char *name) {
    if (!name || !*name)
        return 0;

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i]; i++) {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int export_variable(const char *assignment) {
    char buffer[256];
    strncpy(buffer, assignment, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';

    char *eq = strchr(buffer, '=');
    if (!eq) {
        printf("Invalid export syntax. Use NAME=VALUE\n");
        return 1;
    }

    *eq = '\0';
    char *name = buffer;
    char *value = eq + 1;

    if (!is_valid_name(name)) {
        printf("Invalid variable name: %s\n", name);
        return 1;
    }

    if (setenv(name, value, 1) != 0) {
        perror("setenv");
        return 1;
    }
}
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v task4.c
GNU nano 8.7.1
    for (int i = 1; name[i]; i++) {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int export_variable(const char *assignment) {
    char buffer[256];
    strncpy(buffer, assignment, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';

    char *eq = strchr(buffer, '=');
    if (!eq) {
        printf("Invalid export syntax. Use NAME=VALUE\n");
        return 1;
    }

    *eq = '\0';
    char *name = buffer;
    char *value = eq + 1;

    if (!is_valid_name(name)) {
        printf("Invalid variable name: %s\n", name);
        return 1;
    }

    if (setenv(name, value, 1) != 0) {
        perror("setenv");
        return 1;
    }

    printf("Variable exported successfully.\n");
    printf("%s = %s\n", name, getenv(name));
    return 0;
}

int main(void) {
    export_variable("STUDENT=Shreyaj");
    return 0;
}
```

```
shreyaj@LAPTOP-FT6VVM4D: × + ▾
GNU nano 8.7.1 child.c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    char *student = getenv("STUDENT");

    if (student)
        printf("Child process received STUDENT = %s\n", student);
    else
        printf("Child process: STUDENT not set\n");

    return 0;
}
```

```
shreyaj@LAPTOP-FT6VVM4D: × + ▾
GNU nano 8.7.1 parent.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    setenv("STUDENT", "Shreyaj", 1);
    printf("Parent: STUDENT = %s\n", getenv("STUDENT"));

    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        execl("./child", "child", NULL);
        perror("execl");
        exit(1);
    }

    wait(NULL); // Parent waits for child
    return 0;
}
```